Multi-Tool Coexistence & Advanced Configuration

> **Series:** K8S | **Notebook:** 11 of 12 | **Created:** January 2026 | **Last Updated:** 01/30/2026

Running Dynatrace Alongside Other Monitoring Tools

Many organizations run multiple monitoring tools during migrations or for specialized use cases. This notebook covers patterns for running Dynatrace alongside tools like New Relic, Datadog, or Prometheus without conflicts.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring enabled
Kubernetes Cluster	Dynatrace Operator v1.0+ installed
Knowledge	Completed K8S-01 and K8S-02
Scenario	Running multiple monitoring tools

1. Coexistence Patterns

Understanding Your Options

When running Dynatrace alongside other APM tools, you have two primary approaches:

Approach	Description	Use Case
Infrastructure-Only	Omit `cloudNativeFullStack`	Dynatrace monitors K8s infrastructure; other tool handles APM
Selective APM (Opt-In)	Use namespace selectors	Different tools monitor different workloads

Infrastructure-Only Monitoring

If you omit `oneAgent.cloudNativeFullStack` entirely:

Included	Not Included
Kubernetes cluster health	Application-level APM
Workload metrics	Code-level tracing
Pod/container metrics	Distributed traces
Kubernetes events	OneAgent injection
ActiveGate capabilities	

```yaml
# Infrastructure-only DynaKube (no cloudNativeFullStack)
apiVersion: dynatrace.com/v1beta3
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://your-tenant.live.dynatrace.com/api

  # Only ActiveGate for K8s monitoring - no OneAgent injection
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
    replicas: 2

  # Log monitoring still works without OneAgent
  logMonitoring: {}
```

This is appropriate when:
- Another tool (New Relic, Datadog) handles all APM
- You only need Dynatrace for K8s platform observability
- You want to avoid any agent conflicts

2. Opt-In Mode Configuration

Enable Selective Monitoring

For selective APM where Dynatrace monitors some workloads while other tools monitor others:

Step 1: Disable automatic injection globally

```yaml
metadata:
  annotations:
    feature.dynatrace.com/automatic-injection: "false"
```

Step 2: Add namespace selector to only monitor labeled namespaces

```yaml
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"
```

Step 3: Label namespaces you want Dynatrace to monitor

```bash
# Enable Dynatrace monitoring for specific namespaces
kubectl label namespace checkout dt-monitoring=true
kubectl label namespace payment dt-monitoring=true

# Verify labels
kubectl get namespaces -l dt-monitoring=true
```

Explicit Exclusions (Belt-and-Suspenders)

For extra safety, explicitly exclude other monitoring tool namespaces:

```yaml
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"
        matchExpressions:
          # Explicitly exclude other monitoring namespaces
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values:
              - newrelic
              - datadog
              - prometheus
              - kube-system
```

Namespace Labels Strategy

Label Value	Effect
`dt-monitoring: "true"`	Dynatrace monitors this namespace
No label	Namespace ignored by Dynatrace
`dt-monitoring: "false"`	Explicit opt-out (for documentation)

3. Feature Flags Reference

DynaKube Annotations

Feature flags are set as annotations on the DynaKube metadata:

```
metadata:
  annotations:
    feature.dynatrace.com/&lt;flag-name&gt;: "&lt;value&gt;"
```

Complete Feature Flags Table

Feature Flag	Values	Default	Purpose
`automatic-injection`	`true` / `false`	`true`	Global injection control
`injection-failure-policy`	`fail` / `silent`	`silent`	Pod startup behavior on injection failure
`label-version-detection`	`true` / `false`	`false`	Detect version from K8s labels
`k8s-app-enabled`	`true` / `false`	`false`	Enable K8s application detection
`max-csi-mount-attempts`	`1-10`	`2`	CSI mount retry attempts
`ignore-unknown-state`	`true` / `false`	`false`	Ignore unknown OneAgent state

Recommended Configuration for Coexistence

```
metadata:
  annotations:
    # Enable Kubernetes application detection
    feature.dynatrace.com/k8s-app-enabled: "true"

    # Opt-in mode - only monitor labeled namespaces
    feature.dynatrace.com/automatic-injection: "false"

    # Detect version from Kubernetes labels
    feature.dynatrace.com/label-version-detection: "true"

    # Fail pod if injection fails (instead of silent failure)
    feature.dynatrace.com/injection-failure-policy: "fail"
```

4. Build Version Propagation

Automatic Version Detection from Labels

Dynatrace can automatically detect application versions from Kubernetes labels.

Step 1: Enable the feature flag

```
metadata:
  annotations:
    feature.dynatrace.com/label-version-detection: "true"
```

> **Note:** You can enable this flag immediately. If labels are missing, Dynatrace simply won't have version metadata - no errors occur. When you add labels later, Dynatrace automatically picks them up.

Step 2: Add standard labels to your deployments

```yaml
# In your application Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: checkout-api
  labels:
    app.kubernetes.io/name: checkout-api
    app.kubernetes.io/version: "1.2.3"
    app.kubernetes.io/component: api
    app.kubernetes.io/part-of: ecommerce
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: checkout-api
        app.kubernetes.io/version: "1.2.3"
```

Kubernetes Recommended Labels

Label	Description	Dynatrace Usage
`app.kubernetes.io/name`	Application name	Service name
`app.kubernetes.io/version`	Application version	Version tracking
`app.kubernetes.io/component`	Component type	Service grouping
`app.kubernetes.io/part-of`	Higher-level application	Application grouping
`app.kubernetes.io/instance`	Instance identifier	Instance differentiation

5. Injection Failure Policy

Control Pod Behavior on Injection Failure

By default, if OneAgent injection fails, pods start anyway with silent failure. You can change this:

Policy	Behavior	Use Case
`silent` (default)	Pod starts without instrumentation	Production - availability first
`fail`	Pod fails to start	Non-prod - catch issues early

Enabling Fail Policy

```
metadata:
  annotations:
    feature.dynatrace.com/injection-failure-policy: "fail"
```

When to Use Each Policy

Environment	Recommended Policy	Rationale
Development	`fail`	Catch injection issues immediately
Staging	`fail`	Validate monitoring before production
Production	`silent`	Availability > perfect monitoring

Troubleshooting Injection Failures

If pods fail to start with `fail` policy, check:

```
# Check webhook logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=webhook

# Check CSI driver logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=csi-driver

# Check pod events
kubectl describe pod &lt;pod-name&gt; -n &lt;namespace&gt;
```

6. Log Monitoring Configuration

Understanding the Configuration Structure

> **Important:** Log monitoring has a split configuration:
> - `spec.logMonitoring: {}` - Enables the feature (empty object only)
> - `spec.templates.logMonitoring` - Configures resources and tolerations

Correct Configuration

```yaml
spec:
  # Enable log monitoring (empty object only – no nested properties)
  logMonitoring: {}

  # Resource configuration goes under templates
  templates:
    logMonitoring:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
      tolerations:
        – effect: NoSchedule
          key: node–role.kubernetes.io/control–plane
          operator: Exists
```

Common Mistakes

```yaml
# WRONG - These will fail with validation errors
spec:
  logMonitoring:
    enabled: true          # Not allowed
    resources: ...         # Not allowed here

# CORRECT
spec:
  logMonitoring: {}        # Just empty object
  templates:
    logMonitoring:         # Resources go here
      resources: ...
```

Excluding Namespaces from Log Collection

To exclude namespaces from log collection (e.g., other monitoring tool namespaces), configure log ingest rules in Dynatrace UI:

1. Navigate to **Settings > Log Monitoring > Log ingest rules**
2. Create rules to include/exclude specific namespaces
3. This is done in the UI, not in the DynaKube YAML

```
// Verify which namespaces have Dynatrace-monitored pods
fetch dt.entity.cloud_application_instance
| expand labels = entity.labels
| filter contains(labels, "dynatrace")
| summarize count = count(), by:{entity.name}
| sort count desc
| limit 20
```

```
// Check if version information is being detected
fetch dt.entity.service
| filter isNotNull(service.version)
| fields entity.name, service.version
| sort entity.name asc
| limit 30
```

7. Complete Configuration Example

Production-Ready DynaKube with Coexistence

```yaml
apiVersion: dynatrace.com/v1beta3
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
  labels:
    dynatrace.com/created-by: dynatrace.kubernetes
  annotations:
    # Enable Kubernetes app monitoring
    feature.dynatrace.com/k8s-app-enabled: "true"

    # Opt-in mode for coexistence with other tools
    feature.dynatrace.com/automatic-injection: "false"

    # Enable version detection from K8s labels
    feature.dynatrace.com/label-version-detection: "true"

    # Fail pod if injection fails (use 'silent' for production)
    feature.dynatrace.com/injection-failure-policy: "fail"
spec:
  apiUrl: https://your-tenant.live.dynatrace.com/api
  networkZone: production

  metadataEnrichment:
    enabled: true

  oneAgent:
    hostGroup: production
    cloudNativeFullStack:
      # Opt-in: Only monitor namespaces with this label
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"
        matchExpressions:
          # Explicitly exclude other monitoring namespaces
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values:
              - newrelic
              - datadog
              - kube-system
```

```yaml
    tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/control-plane
        operator: Exists

activeGate:
  capabilities:
    - routing
    - kubernetes-monitoring
  resources:
    requests:
      cpu: 500m
      memory: 1Gi
    limits:
      cpu: 1000m
      memory: 2Gi
  replicas: 2

templates:
  otelCollector:
    imageRef:
      repository: public.ecr.aws/dynatrace/dynatrace-otel-collector
      tag: "0.16.0"  # Pin version - never use 'latest'
    resources:
      requests:
        cpu: 100m
        memory: 256Mi
      limits:
        cpu: 500m
        memory: 512Mi
  logMonitoring:
    resources:
      requests:
        cpu: 100m
        memory: 256Mi
      limits:
        cpu: 500m
        memory: 512Mi
    tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/control-plane
        operator: Exists

# Enable log monitoring (resources configured above)
logMonitoring: {}

telemetryIngest:
  protocols:
```

```
    - otlp
  serviceName: telemetry-ingest
```

Summary

In this notebook, you learned:

- **Coexistence patterns** for running Dynatrace with other monitoring tools
- **Opt-in mode** using namespace selectors and automatic-injection flag
- **Feature flags reference** for DynaKube annotations
- **Build version propagation** from Kubernetes labels
- **Injection failure policy** for controlling pod startup behavior
- **Log monitoring configuration** with the split spec structure
- **Complete production configuration** for multi-tool environments

---

Next Steps

Next Notebook	Topic
K8S-12: Specialized Monitoring	NGINX Ingress, CSI Driver tuning
K8S-09: Troubleshooting	Diagnosing common issues

References

- DynaKube Feature Flags
- Namespace Selector Configuration
- Kubernetes Recommended Labels

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.